Perceptron Learning Algorithm

Dinanath Padhya*, Krishna Acharya†
*THA078BEI010 †THA078BEI020
Department of Electronics, Communication and Information Engineering
Thapathali Campus, IOE, Tribhuvan University, Nepal

***Abstract*—This report presents a comprehensive exploration of the Perceptron Learning Algorithm applied to fundamental logic functions, including AND, OR, and NOT gates, and examines its inherent limitation with the XOR operation. Beginning with a detailed theoretical background, we introduce the various activation functions and perceptron model. The Heaviside activation function, followed by the learning rule that iteratively adjusts weights and bias based on prediction errors. We also explore about sigmod activation function. We implemented it in Python using NumPy, the algorithm was trained on truth-table datasets, achieving rapid convergence for linearly separable functions. Quantitative results are provided, including final weight vectors, biases, epoch counts, and visualized decision boundaries. In contrast, the XOR gate training process fails to converge, underscoring the necessity of non-linear architectures for non-linearly separable problems. We supplement the previous experimental findings of XOR inseparability and reference historical work by Minsky and Papert. Concluding remarks highlight the perceptron utility for simple classification tasks and its limitations, motivating the progression to multi-layer networks and non-linear activation functions for complex problem domains.**

***Index Terms*—Activation Function, Linear Separability, Logic Gates, Neural Networks, Perceptron, XOR Problem**

I. Introduction

A. Background

Neural networks are computational models inspired by biological neural systems. Among the foundational components of neural networks is the perceptron, a simple yet powerful model introduced by Frank Rosenblatt in 1958. [**?**] The perceptron emulates a biological neuron by taking multiple inputs, applying corresponding weights, computing a weighted sum, and passing it through an activation function to generate an output. As the earliest form of a supervised machine learning algorithm, the perceptron laid the groundwork for binary classification tasks involving linearly separable data.

B. Literature Review

The perceptron algorithm has played a crucial role in the evolution of machine learning and artificial intelligence. Rosenblatt's original work demonstrated its potential for pattern recognition and established a theoretical framework for neural computation. However, Minsky and Papert's seminal book *Perceptrons* (1969) exposed critical limitations, most notably its inability to solve problems involving non-linearly separable data, such as the XOR function. [**?**] This criticism led to a temporary decline in neural network research. The resurgence came in the 1980s with the development of multi-layer perceptrons and the backpropagation algorithm, enabling networks to learn complex patterns. Today, while the perceptron is no longer state-of-the-art, it remains an essential educational tool and a stepping stone toward understanding more sophisticated neural architectures.

C. Objectives

This laboratory experiment aims to:

- Implement the Perceptron Learning Algorithm from first principles.
- Demonstrate the algorithm's capability to learn linearly separable logical functions (AND, OR gates).
- Illustrate the perceptron's limitation in solving non-linearly separable functions (XOR gate).
- Develop a deeper understanding of the theoretical constraints inherent in single-layer perceptron models.

II. Methodology

A. Implementation Environment

The Perceptron Learning Algorithm was implemented in Python using NumPy for efficient numerical computations. The code is structured to define the perceptron model, activation functions, and training loop, allowing for straightforward experimentation with different logic gates.

B. Theoretical Basis

Perceptron Learning Algorithm: The Perceptron Learning Agorithm(PLA) is a supervised learning algorithm for binary classifiers. It is a type of linear classifier, meaning that it makes its predictions based on a linear predictor function combining a set of weights with the feature vector. It takes an input, aggregates it and returns 1 if the aggregated sum is greater than a certain threshold, otherwise it returns 0.

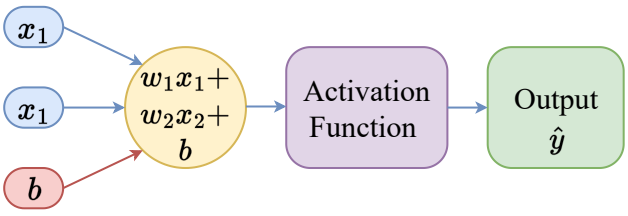


Fig. 1. Perceptron Learning Algorithm

The Perceptron Learning Algorithm is a simple and efficient algorithm for training a single-layer neural network. It is based on the idea of adjusting the weights of the network based on the error between the predicted output and the actual output. The algorithm iteratively updates the weights until the error is minimized.

Activation Functions: An activation function is a mathematical function that determines the output of a neural network node. It takes the weighted sum of the inputs and applies a non-linear transformation to produce the output. The most common activation functions are:

Unit Step Function

The unit step function is a simple activation function that outputs 1 if the input is greater than or equal to a certain threshold, and 0 otherwise. It is defined as:

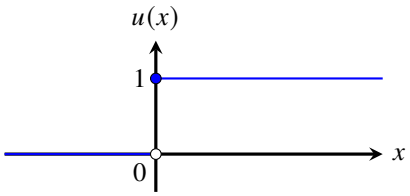


Fig. 2. Unit Step Function

$$u(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \tag{1}$$

Sigmoid Function

The sigmoid function is a smooth, S-shaped curve that maps any input to a value between 0 and c which is defined as:

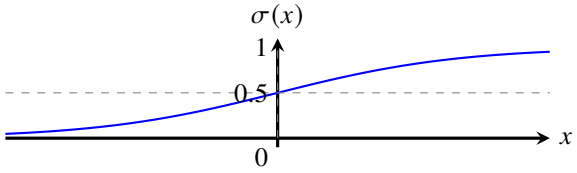


Fig. 3. Sigmoid Activation Function

$$f(x) = \frac{c}{a + be^{-x}} \tag{2}$$

where a and b are constants that determine the shape of the curve. The sigmoid function is commonly used in neural networks because it is differentiable and has a well-defined gradient.

Dataset

In this lab, perceptron was trained to learn the AND, OR and XOR logic gates. The dataset consists of 4 samples for each gate, with 2 input features and 1 output label. The input features are binary values (0 or 1) representing the inputs to the gate, and the output label is also a binary value representing the output of the gate. The dataset is as follows:

III. Experiment & Model Implementation

The main components include the sigmoid activation function, the perceptron model, and the training loop. The following code snippets illustrate the key parts of the implementation.

Input		Output		
X_1	X_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

TABLE I
Truth tables for AND, OR, and XOR logic gates

A. Sigmoid Function

The sigmoid function is used as an activation function to map the input to a value between 0 and 1.

```
def sigmoid(x):
    return 1 / (1 + math.exp(-x))
```

Listing 1. Sigmoid Function

B. Activation Function

The activation function applies the sigmoid and then thresholds the output to produce a binary result.

```
def activation(z):
    out = sigmoid(z)
    if out >= 0.5:
        return 1
    else:
        return 0
```

Listing 2. Activation Function

C. Perceptron Model

The perceptron computes the weighted sum of the inputs.

```
def perceptron(w1, w2, x1, x2):
    z = w1*x1 + w2*x2
    return z
```

Listing 3. Perceptron Model

D. Training the Perceptron

The perceptron is trained using the perceptron learning rule, updating weights based on the error.

```
def train_perceptron(inputs, outputs):
    global w0, w1, b
    w0 = random.uniform(-1, 1)
    w1 = random.uniform(-1, 1)
    b = random.uniform(-1, 1)
    epochs = 1000
    learning_rate = 0.1
    for epoch in range(epochs):
        total_error = 0
        for i in range(len(inputs)):
            A, B = inputs[i]
            target_output = outputs[i]
            predicted = activation(w0*A + w1*B +
                b)
            error = target_output - predicted
            w0 += error * A * learning_rate
            w1 += error * B * learning_rate
            b +=  error * learning_rate
            total_error += abs(error)
        if total_error == 0:
            print(f"Training completed in
                {epoch+1} epochs")
            break
```

Listing 4. Training Loop

E. Testing the Perceptron

After training, the perceptron can be tested on the input data.

```
def test_perceptron(inputs):
    for i in range(len(inputs)):
        A, B = inputs[i]
        output = activation(w0*A + w1*B + b)
        print(output)
```

Listing 5. Testing the Perceptron

IV. Results

We trained the perceptron on the AND, OR, and XOR datasets for 1000 epochs. The AND and OR gates converged successfully, while the XOR gate did not converge due to its non-linearly separable nature.

```
train_perceptron(inputs, outputs)
print(w0, w1, b)
test_perceptron([(1, 1)])
# Output:
# 0.0s
# Training completed in 2 epochs
# 0.5910945991469214 -0.10140749810229646
    0.048412417633727806
```

Listing 6. Training Perceptron on AND Gate

```
train_perceptron(inputs, outputs_or)
print(w0, w1, b)
test_perceptron([(0, 1)])
# Output:
# 0.0s
# Training completed in 4 epochs
# -0.674110728071617 0.7278901116381135
    -0.5947259315162406
```

Listing 7. Training Perceptron on OR Gate

```
train_perceptron(inputs, outputs_xor)
print(w0, w1, b)
test_perceptron([(1, 1)])
# Output:
# 0.0s
# Training did not converge.
# 0.015820858948757927 0.05471201366852324
    0.0964247823885859
```

Listing 8. Training Perceptron on XOR Gate

Discussion

A. Why the XOR Problem Cannot Be Solved by a Single-Layer Perceptron

The XOR (exclusive OR) gate is a classic example of a problem that cannot be solved by a single-layer perceptron. This is because the XOR function is not linearly separable; there is no straight line that can separate the output classes in the input space. A single-layer perceptron can only learn problems where such a linear boundary exists, like AND or OR gates. As a result, the perceptron fails to correctly classify all possible inputs for the XOR gate.

B. Role of the Logistic Function

The logistic function, also known as the sigmoid function, is commonly used as an activation function in neural networks. It introduces non-linearity into the model, allowing the perceptron to learn more complex patterns than a simple linear model. However, even with a non-linear activation function like the logistic function, a single-layer perceptron still cannot solve the XOR problem due to its inherent linear structure. To overcome this limitation, multi-layer perceptrons (MLPs) are used, which stack multiple layers of perceptrons and can learn non-linear decision boundaries, making it possible to solve the XOR problem and other complex tasks.

V. Conclusion

The Perceptron Learning Algorithm is a foundational concept in machine learning and neural networks. It provides a simple yet effective way to learn linear decision boundaries. However, its limitations in handling non-linearly separable data highlight the need for more advanced architectures, such as multi-layer perceptrons and deep learning models. The implementation of the perceptron learning algorithm in Python demonstrates its practical application to basic logic gates. Future work could involve exploring more complex datasets and architectures to further enhance the capabilities of neural networks.